

SAT- und MaxSAT-Enkodierung des Art Gallery Problems

Methodik, Implementierung und Benchmark-Ergebnisse

Elena Berschauer

24. Juni 2026

Lehrstuhl für Rechnerarchitektur

Betreuer: Mathias Fleury

Art Gallery Problem

Das Art Gallery Problem

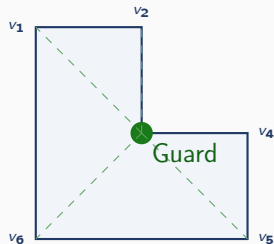
Problem der Computational Geometry

Wie viele Guards werden benötigt, um ein Polygon vollständig zu überwachen?

- Klassisches Problem seit den 1970er Jahren
- Chvátals Art-Gallery-Theorem: $\lfloor \frac{n}{3} \rfloor$ Guards genügen
- Exakte Lösungen für große Instanzen schwierig

Fragestellung dieser Arbeit

Wie lässt sich das AGP als SAT bzw. MaxSAT modellieren?



Beispiel: Ein Guard genügt für das L-Polygon.

Diskretisierung & Kandidaten

Vom kontinuierlichen AGP zur Diskretisierung

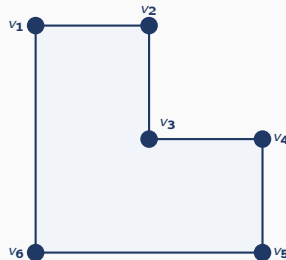
Problem

Das AGP ist kontinuierlich: Guards können prinzipiell überall im Polygon stehen.

Diskretisierung

- Endliche Kandidatenmenge P
- Guards nur auf Punkten aus P
- Witnesses ebenfalls aus P

$$P = \{p_1, \dots, p_n\}$$



Kontinuierliches Polygon $\rightarrow$ endliche Punktmenge

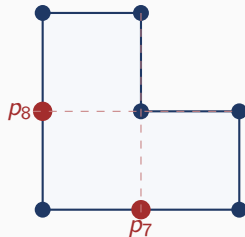
Vorgehen

1. Polygon-Ecken P_0
2. Sichtbare Punktpaare verbinden
3. Neue Schnittpunkte aufnehmen

$$P_{r+1} = P_r \cup \Delta_r$$

Δ_r = neue gültige Schnittpunkte

Iteration bis keine neuen Punkte entstehen.



L-Polygon: 6 Eckpunkte + 2 neue
Schnittpunkte

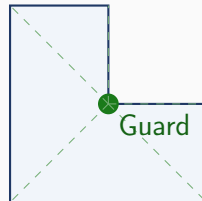
Definition

$$V[i][j] = 1$$

genau dann, wenn

$$\overline{p_i p_j} \subseteq \overline{P}$$

Die Geometrie wird vollständig durch die Sichtbarkeitsmatrix V beschrieben.



$$V = \begin{pmatrix} 1 & 1 & 1 & 0 & \dots \\ 1 & 1 & 1 & 0 & \dots \\ 1 & 1 & 1 & 1 & \dots \\ \vdots & & & & \end{pmatrix}$$

$$V \implies \text{SAT-Encoding}$$

SAT-Encoding

Guard-Variable

Für jeden Kandidaten $p_i \in P$:

$$g_i \in \{0, 1\}, \quad g_i = 1 \iff p_i \text{ wird als Guard eingesetzt}$$

Coverage-Klausel (pro Witness p_j)

$$\forall j : \bigvee_{V[i][j]=1} g_i$$

Mindestens ein Guard, der p_j sieht, muss aktiv sein.

Coverage allein reicht nicht

Triviale Lösung: alle $g_i = 1$ erfüllt alle Klauseln. $\Rightarrow$ Kardinalitätsbeschränkung nötig.

Kardinalitätsbeschränkung: $\sum g_i \leq k$

- SAT kennt keine Summation — nur boolesche Disjunktionen
- Die Schranke muss also über Klauseln simuliert werden

Sequential Counter Encoding

Simuliert einen Zähler über Hilfsvariablen $s_{i,j}$.

$\mathcal{O}(n \cdot k)$ zusätzliche Variablen und Klauseln — skaliert linear in n und k , allgemein einsetzbar.

Sequential Counter: Hilfsvariable & Initialisierung

Hilfsvariable

$s_{i,j} = 1 \iff$ unter den ersten i Kandidaten sind mindestens j Guards aktiv

Beispiel: $s_{5,2} = 1$ bedeutet: unter $g_1, \dots, g_5$ sind mindestens 2 Guards aktiv.

Initialisierung ($i = 1$): Äquivalenz $s_{1,1} \leftrightarrow g_1$

$$g_1 = 1 \Rightarrow s_{1,1} = 1$$

Implikation: $g_1 \rightarrow s_{1,1}$

$$s_{1,1} = 1 \Rightarrow g_1 = 1$$

Implikation: $s_{1,1} \rightarrow g_1$

$$\neg g_1 \vee s_{1,1}$$

$$\neg s_{1,1} \vee g_1$$

Beide Klauseln zusammen erzwingen die Äquivalenz — ohne Richtung 2 könnte der Solver $s_{1,1}$ frei setzen.

Herleitung: Monotonie & Inkrement

Monotonie

Regel: Sind unter den ersten $i-1$ Kandidaten schon $\geq j$ Guards aktiv, gilt das auch für die ersten i .

Implikation: $s_{i-1,j} \rightarrow s_{i,j} \quad (a \rightarrow b \equiv \neg a \vee b)$

$$\neg s_{i-1,j} \vee s_{i,j}$$

Inkrement

Regel: Sind unter den ersten $i-1$ Kandidaten $\geq j-1$ Guards aktiv *und* wird g_i zusätzlich gewählt, sind es $\geq j$.

Implikation: $s_{i-1,j-1} \wedge g_i \rightarrow s_{i,j} \quad ((a \wedge b) \rightarrow c \equiv \neg a \vee \neg b \vee c)$

$$\neg s_{i-1,j-1} \vee \neg g_i \vee s_{i,j}$$

Obergrenze

Regel: Mehr als k Guards sind verboten. Sind unter den ersten $i-1$ Kandidaten schon k Guards aktiv ($s_{i-1,k} = 1$), darf g_i nicht mehr aktiviert werden.

Implikation: $s_{i-1,k} \rightarrow \neg g_i \quad \equiv \quad \neg(s_{i-1,k} \wedge g_i)$

$$\neg s_{i-1,k} \vee \neg g_i$$

Das ist die eigentliche Schranke $\sum_i g_i \leq k$.

Iteratives Lösen mit Kissat

1. Starte mit $k = 1$
2. Erzeuge CNF: Coverage-Klauseln $\wedge$ Sequential Counter ($\leq k$)
3. Kissat löst die Instanz
4. Falls **UNSAT**: $k \leftarrow k + 1$, zurück zu Schritt 2
5. Erstes **SAT**-Ergebnis $\Rightarrow k$ ist minimal

MaxSAT-Encoding

Motivation

Iteratives SAT: im schlimmsten Fall $k_{\min}$ Solver-Aufrufe, jeder UNSAT-Nachweis teurer als der vorherige.

MaxSAT formuliert Minimierung direkt:

- **Hard Clauses:** müssen erfüllt sein — Coverage-Klauseln
- **Soft Clauses:** sollen erfüllt sein, mit Gewicht — Verletzungen werden minimiert

AGP als MaxSAT

$$\text{Hard: } \bigvee_{i: V[i][j]=1} g_i \quad \forall j \qquad \text{Soft (Gewicht 1): } \neg g_i \quad \forall i$$

Jede verletzte Soft Clause ($g_i = 1$) kostet 1.

Solver minimiert Kosten = Anzahl aktiver Guards.

Kein Sequential Counter, kein iteratives k .

Evaluation

- **Solver**

- SAT: Kissat
- MaxSAT: MaxHS

- **Instanzfamilien**

- comb: kontrolliert wachsendes Optimum
- random: zufällige Sichtbarkeitsstrukturen

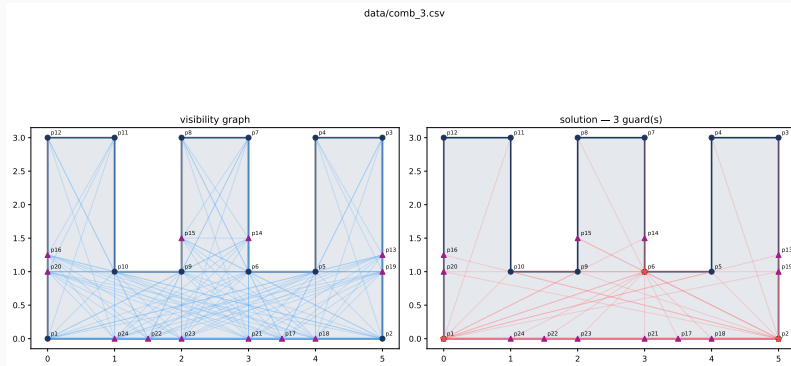
- **Kandidatengenerierung**

- 0, 1 und 2 Runden

- **Messung**

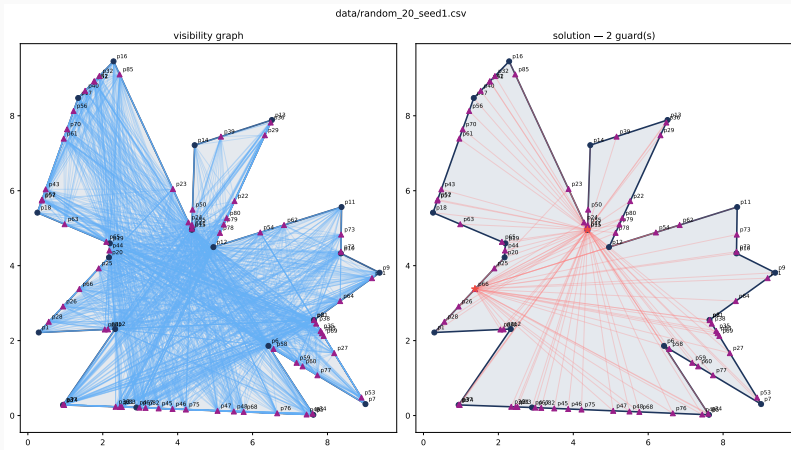
1. Vorverarbeitung (Kandidaten, Sichtbarkeit, CNF/WCNF)
2. Solverlaufzeit auf gecachten Instanzen

Ergebnisse

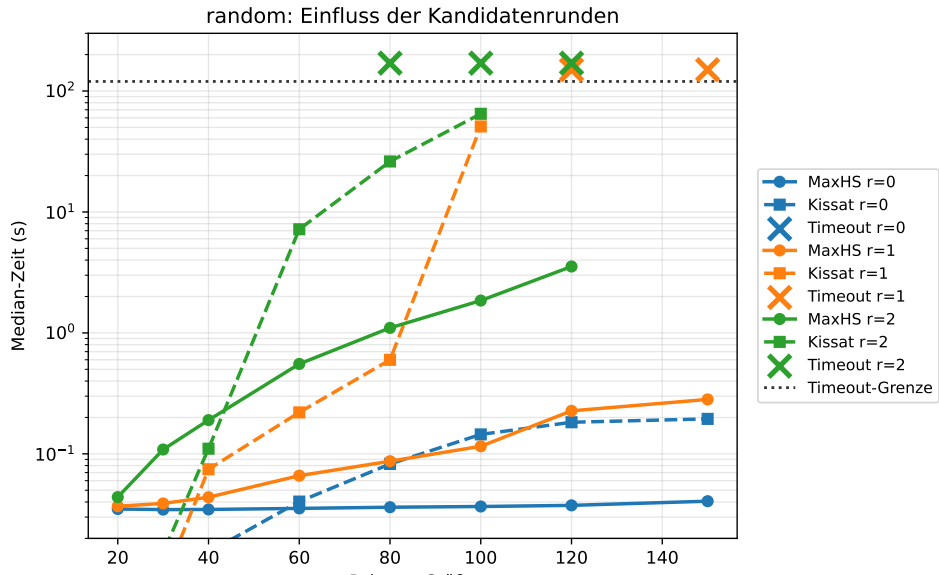


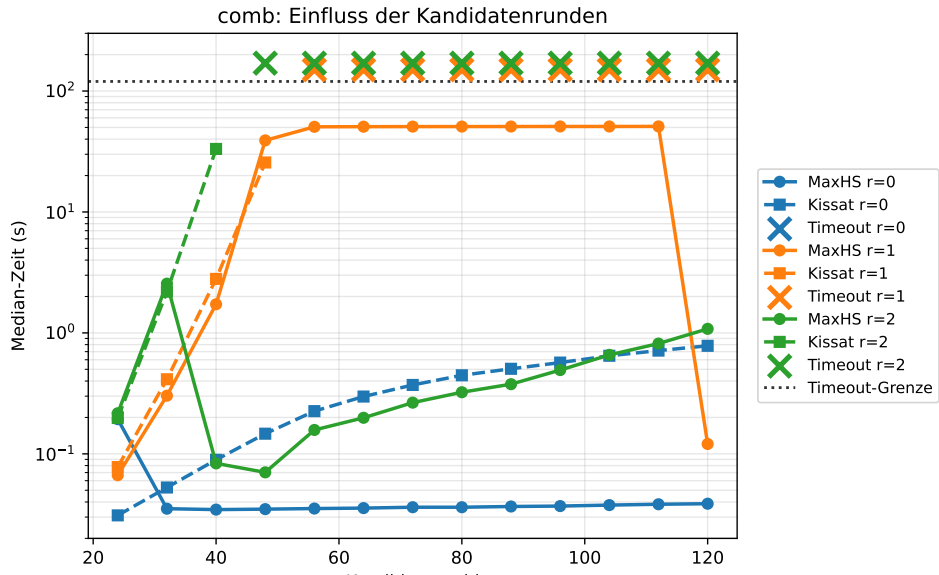
comb_3: optimale Lösung mit $k_{\min} = 3$ Guards

Beispiellösungen



random_20_seed1: optimale Lösung mit $k_{\min} = 2$ Guards





Diskussion

Beobachtungen

- Zusätzliche Runden erhöhen die Kandidatenzahl stark
- Das Optimum verändert sich meist kaum
- Kissat verliert Zeit in aufeinanderfolgenden UNSAT-Nachweisen
- MaxHS vermeidet die iterative Suche nach k

Skalierung

- Vorverarbeitung dominiert bei großen Instanzen
- Große Optima sind besonders schwierig für SAT

Fazit & Ausblick

Zusammenfassung

- AGP erfolgreich als SAT und MaxSAT modelliert
- Sequential Counter ermöglicht exakte Kardinalitätsbeschränkung
- MaxHS vermeidet die iterative Suche über k
- Zusätzliche Kandidatenrunden erhöhen die Instanzgröße stark
- Kandidatengenerierung dominiert bei großen Instanzen

Ausblick

- **Kandidatengenerierung** ist Flaschenhals bei großen Instanzen (random_30: Vorverarbeitung sprengt Zeitlimit) $\Rightarrow$ Optimierungspotenzial als nächster Schritt
- **Rundenvergleich**: 1 vs. 2 vs. mehr Runden — bessere Optima?

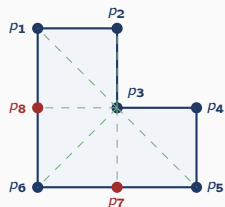
Pipeline-Überblick

Pipeline-Überblick



- **Candidates:** Eckpunkte + relevante Kantenschnittpunkte
- **Visibility:** Sichtbarkeitsmatrix V über Sub-Segment-Dekomposition
- **Encoding:** Fokus dieses Vortrags
- **Solving:** Kissat (iterativ) vs. MaxHS (direkt)

Anhang: Sichtbarkeitsmatrix V — L-Polygon ($|P| = 8$)



Guard p_3 sieht alle

$|P| = 8$
 (p_1 – p_6 Ecken, p_7, p_8
 Schnittpunkte)

$$V[i][j] = 1 \iff \overline{p_i p_j} \subseteq \bar{P}$$

	p_1	p_2	p_3	p_4	p_5	p_6	p_7	p_8
p_1	1	1	1	0	1	1	1	1
p_2	1	1	1	0	0	1	1	1
p_3	1	1	1	1	1	1	1	1
p_4	0	0	1	1	1	1	1	1
p_5	1	0	1	1	1	1	1	1
p_6	1	1	1	1	1	1	1	1
p_7	1	1	1	1	1	1	1	1
p_8	1	1	1	1	1	1	1	1

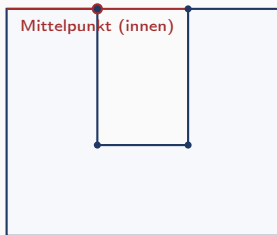
p_3, p_6, p_7, p_8 : vollständige Zeile $\Rightarrow$ sehen alle 8 Punkte

$V[1][4] = 0$: $p_1 \rightarrow p_4$ verlässt das Polygon

Anhang: Sub-Segment-Dekomposition

Naiver Mittelpunktstest: Falsch positiv

Segment von $(0, 5)$ zu $(4, 5)$ im
U-Polygon:

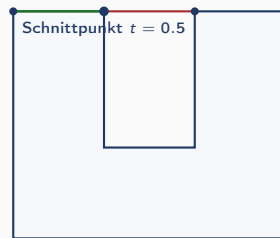


Mittelpunkt $(2, 5)$ liegt innen $\Rightarrow$ sichtbar

Aber: Segment geht durch Lücke!

Sub-Segment-Dekomposition

Zerlegung an allen Schnittpunkten mit
dem Polygonrand, Mittelpunkt *jedes*
Teilsegments wird geprüft:

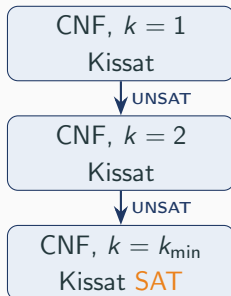


Teilsegment 2, Mittelpunkt bei $t = 0.75$:

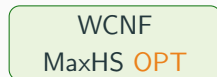
$(3, 5)$ liegt außen

$\Rightarrow$ **nicht sichtbar** — korrekt

Kissat (iterativ) vs. MaxHS (direkt)



Kissat: mehrere kleine CNF-Instanzen
Vorteil bei kleinem $k_{\min}$ (wenige
UNSAT-Runden)



MaxHS: eine WCNF-Instanz
Vorteil bei großem $k_{\min}$ (viele teure
UNSAT-Nachweise entfallen)

WCNF-Format: konkretes Beispiel (L-Polygon, $|P| = 8$)

c AGP MaxSAT, L-Polygon

h 1 2 3 5 6 7 8 0 % Cov. p_1

h 1 2 3 6 7 8 0 % Cov. p_2

h 1 2 3 4 5 6 7 8 0 % Cov. p_3

...

1 -1 0 % Soft: $\neg g_1$

1 -2 0 % Soft: $\neg g_2$

...

1 -8 0 % Soft: $\neg g_8$

Legende

h = Hard Clause (Coverage)

1 = Soft Clause, Gewicht 1

MaxHS minimiert:

$$\sum_i 1[g_i = 1]$$

= Anzahl aktiver Guards

Lösung: MaxHS setzt $g_3 = 1$,
alle anderen = 0

$\Rightarrow$ Kosten = 1 = $k_{\min}$

(auch g_6, g_7, g_8 optimal)

Einschränkung: Was heißt „optimal“?

SAT/MaxSAT liefert

Optimale Lösung bezüglich des Kandidatensatzes C .

Aber nicht

Das globale Optimum des kontinuierlichen AGP.

Grund

Das kontinuierliche AGP ist $\exists\mathbb{R}$ -vollständig; optimale Guards können irrationale Koordinaten besitzen.